

PKI & PKC

Information Security

BS Computer Science

BCS4A&B



Instructor: Sabghat Ullah Khan
Department of Computer Science
CUI Islamabad
Date: November 24, 2025

Contents

1	Introduction to Public Key Infrastructure	3
1.1	What is PKI?	3
1.2	Why Do We Need PKI?	3
1.3	Real-World Applications of PKI	4
2	Core Components of PKI	4
2.1	1. Certificate Authority (CA)	4
2.2	2. Registration Authority (RA)	5
2.3	3. Digital Certificates	5
2.4	4. Certificate Repository	6
2.5	5. Certificate Revocation System	6
2.6	6. Key Management System	7
3	PKI Trust Models	7
3.1	1. Hierarchical Trust Model (Tree Structure)	7
3.2	2. Distributed Trust Model (Web of Trust)	7
3.3	3. Bridge Trust Model	8
4	Certificate Lifecycle	8
4.1	Phase 1: Certificate Request and Issuance	8
4.2	Phase 2: Certificate Validation	9
4.3	Phase 3: Certificate Renewal	9
4.4	Phase 4: Certificate Revocation	10
5	Implementing PKI Operations in Python	10
5.1	Example 1: Complete Self-Signed Certificate Creation	10
5.2	Example 2: Certificate Validation	12
5.3	Example 3: Create Certificate Revocation List (CRL)	13
6	PKI in Real-World Scenarios	15
6.1	Scenario 1: HTTPS/TLS in Web Browsers	15
6.2	Scenario 2: Email Security with S/MIME	15
6.3	Scenario 3: Code Signing	16
7	PKI Security Considerations	16
7.1	Threats to PKI	16
7.2	Best Practices	17
8	Practice Questions and Activities	17
8.1	Programming Exercises	17
8.2	Lab Activity: HTTPS Certificate Analysis	19
9	Introduction to Public-Key Certificates	21
9.1	What is a Public-Key Certificate?	21
9.2	Certificate Structure Notation	21
9.3	Key Properties of Certificates	22

10 Certificate Chains and Hierarchy	22
10.1 The Trust Problem	22
10.2 Solution: Certificate Chains	22
11 Implementing X.509 Certificates in Python	22
11.1 Step 1: Generating an RSA Private Key	22
11.2 Step 2: Creating a Certificate Signing Request (CSR)	23
11.3 Step 3: Loading an Existing Private Key	24
12 Certificate Revocation	24
12.1 Why Revocation is Necessary	24
12.2 Certificate Revocation Lists (CRLs)	24
13 Practice Lab Activities	25
13.1 Lab Activity 1: Certificate Validation	25
13.2 Lab Activity 2: Certificate Revocation List	25

1 Introduction to Public Key Infrastructure

1.1 What is PKI?

Public Key Infrastructure (PKI) is a comprehensive framework that:

1. Manages digital certificates and public-key encryption
2. Enables secure electronic transfer of information
3. Provides authentication, confidentiality, integrity, and non-repudiation
4. Supports various security services across networks

Simple Analogy: Think of PKI as a digital passport system for the Internet. Just as a government issues passports to verify citizens' identities when traveling, PKI issues digital certificates to verify entities' identities online.

1.2 Why Do We Need PKI?

The Trust Problem in Cryptography:

In asymmetric cryptography, we face a critical challenge:

- Alice has a public key and a private key
- Alice shares her public key openly
- **Problem:** How does Bob know that the public key he received actually belongs to Alice and not an attacker?

Without PKI: An attacker (Mallory) could perform a Man-in-the-Middle (MITM) attack:

1. Mallory intercepts Alice's public key sent to Bob
2. Mallory sends her own public key to Bob, claiming it's Alice's
3. Bob encrypts messages with Mallory's key thinking it's Alice's
4. Mallory can decrypt, read, and re-encrypt messages

With PKI: This attack is prevented because:

- Certificate Authorities digitally sign certificates
- Bob can verify Alice's certificate using the CA's public key
- Any tampering with the certificate is detected

1.3 Real-World Applications of PKI

1. **HTTPS/SSL/TLS:** Secure web browsing (the padlock icon in browsers)
2. **Email Security:** S/MIME for encrypted and signed emails
3. **Code Signing:** Verifying software publishers and integrity
4. **VPN Authentication:** Secure remote access to corporate networks
5. **Digital Signatures:** Legal documents, e-governance, banking
6. **Smart Cards:** Employee ID cards, payment cards
7. **IoT Security:** Authenticating devices in smart homes and industries

2 Core Components of PKI

PKI consists of several interconnected components working together:

2.1 1. Certificate Authority (CA)

Definition: A trusted third party that issues and manages digital certificates.

Key Responsibilities:

- Issue digital certificates after verifying identities
- Digitally sign certificates using its private key
- Maintain certificate records
- Publish Certificate Revocation Lists (CRLs)
- Manage certificate lifecycle

Types of Certificate Authorities:

1. Root CA:

- Top-level CA in the trust hierarchy
- Self-signed certificate
- Pre-installed in browsers/operating systems
- Examples: DigiCert, Let's Encrypt, GlobalSign

2. Intermediate CA:

- Certified by Root CA
- Issues end-entity certificates
- Provides isolation (protects Root CA's private key)

3. Subordinate CA:

- Lower in hierarchy
- Specialized purposes (e.g., different departments)

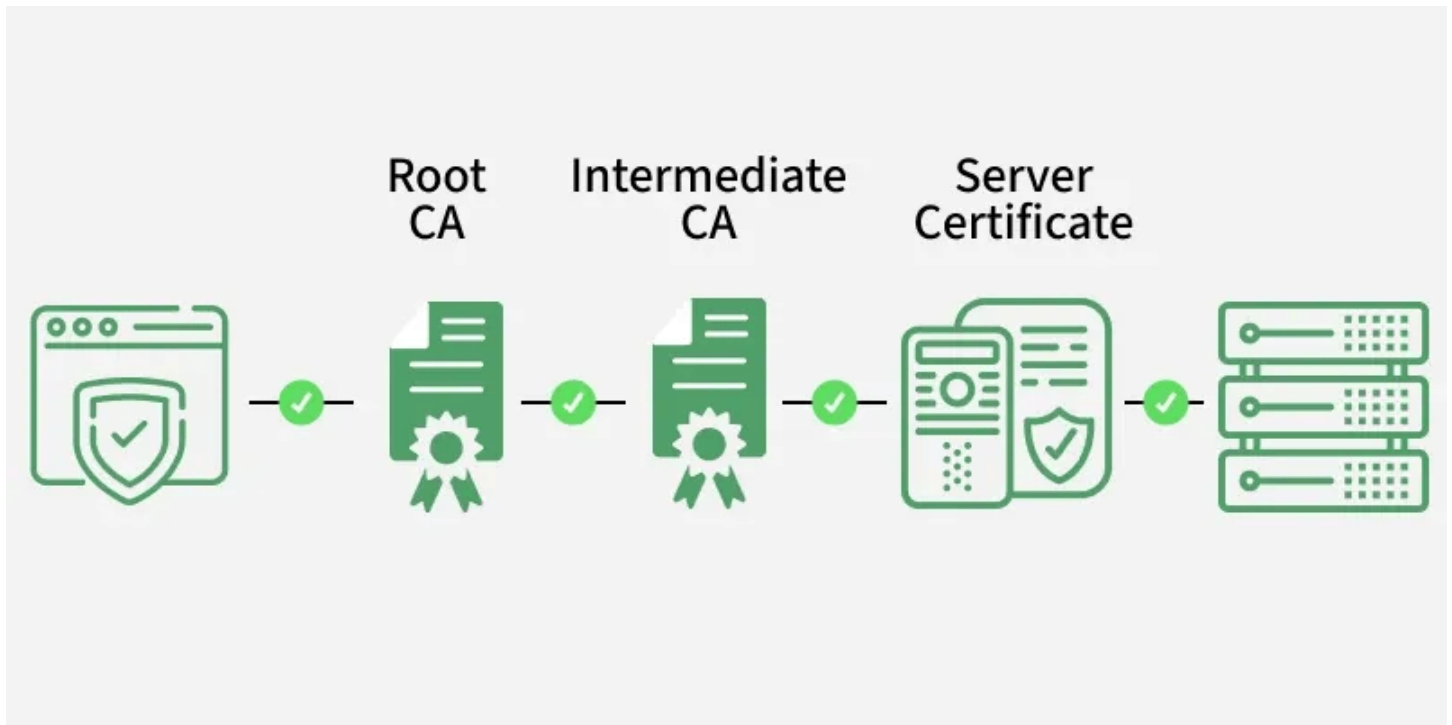


Figure 1: Types of CA

2.2 2. Registration Authority (RA)

Definition: An intermediary that verifies certificate requests before forwarding them to the CA.

Key Responsibilities:

- Verify identity of certificate requesters
- Validate ownership of domain names or email addresses
- Authenticate requests using documents, face-to-face verification, etc.
- Forward approved requests to CA
- Initiate certificate revocation requests

Note: In smaller PKI implementations, the CA may perform RA functions directly.

2.3 3. Digital Certificates

Definition: An electronic document that binds a public key to an identity.

X.509 Certificate Format (Standard):

- **Version:** X.509 version number (typically v3)
- **Serial Number:** Unique identifier for the certificate
- **Signature Algorithm:** Algorithm used to sign (e.g., RSA with SHA-256)
- **Issuer:** Name of the CA that issued the certificate
- **Validity Period:** Not Before and Not After dates

- **Subject:** Name of the certificate holder (person, organization, server)
- **Subject Public Key:** The actual public key and algorithm
- **Extensions:** Additional attributes (e.g., Key Usage, Subject Alternative Names)
- **Signature:** CA's digital signature on the certificate

2.4 4. Certificate Repository

Definition: A publicly accessible database that stores certificates and CRLs.

Functions:

- Store issued certificates
- Publish Certificate Revocation Lists
- Allow users to retrieve certificates
- Enable certificate validation

Common Implementations:

- LDAP (Lightweight Directory Access Protocol) servers
- HTTP/HTTPS web servers
- X.500 directory services

2.5 5. Certificate Revocation System

Purpose: Invalidate certificates before their expiration date.

Methods:

1. Certificate Revocation List (CRL):

- List of revoked certificate serial numbers
- Published periodically by CA
- Clients download and check before trusting certificates
- *Limitation:* Can be large and slow to propagate

2. Online Certificate Status Protocol (OCSP):

- Real-time certificate status checking
- Client queries OCSP responder for specific certificate
- Receives response: Good, Revoked, or Unknown
- *Advantage:* Faster and more efficient than CRLs

3. OCSP Stapling:

- Server obtains OCSP response and "staples" it to the certificate
- Reduces client load and improves performance
- Enhances privacy (client doesn't contact OCSP server)

2.6 6. Key Management System

Purpose: Secure generation, storage, distribution, and destruction of cryptographic keys.

Key Functions:

- **Key Generation:** Create strong random keys
- **Key Storage:** Hardware Security Modules (HSMs), secure databases
- **Key Backup:** Prevent data loss if keys are lost
- **Key Recovery:** Retrieve encrypted data if keys are lost
- **Key Archival:** Long-term storage for compliance
- **Key Destruction:** Securely delete keys when no longer needed

3 PKI Trust Models

3.1 1. Hierarchical Trust Model (Tree Structure)

Structure:

- Single Root CA at the top
- Intermediate CAs below
- End-entity certificates at the bottom

Characteristics:

- Most common model (used in web browsers)
- Clear chain of trust
- Root CA must be absolutely trusted

Example:

```
Root CA (DigiCert Global Root)
|
+-- Intermediate CA (DigiCert SHA2 Secure Server CA)
    |
    +-- End Entity (www.example.com)
```

3.2 2. Distributed Trust Model (Web of Trust)

Structure:

- No central authority
- Users sign each other's certificates
- Trust is based on personal relationships

Characteristics:

- Used in PGP (Pretty Good Privacy)
- Decentralized approach
- Trust levels: Unknown, Marginal, Full

3.3 3. Bridge Trust Model

Structure:

- Multiple independent PKI hierarchies
- Connected by a Bridge CA
- Enables cross-certification

Use Case: Government and corporate PKIs that need to interoperate

4 Certificate Lifecycle

4.1 Phase 1: Certificate Request and Issuance

Step-by-Step Process:**1. Key Pair Generation:**

- End entity generates public-private key pair
- Private key remains secure with the entity

2. Certificate Signing Request (CSR) Creation:

- Entity creates CSR containing public key and identity information
- CSR is signed with entity's private key (proves key ownership)

3. Identity Verification (via RA):

- Domain validation (for websites)
- Organization validation (verify company registration)
- Extended validation (rigorous identity checks)

4. Certificate Generation:

- CA creates certificate with all required fields
- CA signs certificate with its private key

5. Certificate Distribution:

- Certificate is sent to the requester
- Published in certificate repository

4.2 Phase 2: Certificate Validation

Validation Steps (performed by relying parties):

1. Verify Signature:

- Decrypt certificate signature using CA's public key
- Compare with hash of certificate contents

2. Check Validity Period:

- Ensure current date is within Not Before and Not After dates

3. Verify Certificate Chain:

- Trace back to trusted root CA
- Validate each intermediate certificate

4. Check Revocation Status:

- Query CRL or OCSP
- Ensure certificate hasn't been revoked

5. Verify Purpose:

- Check Key Usage extensions
- Ensure certificate is used for intended purpose

4.3 Phase 3: Certificate Renewal

When to Renew:

- Before expiration (typically 30-90 days before)
- After key compromise (requires new key pair)
- When identity information changes

Renewal Process:

1. Generate new CSR (may reuse key pair or generate new one)
2. Submit renewal request to CA
3. CA verifies identity (may be faster for existing customers)
4. New certificate issued with new validity period

4.4 Phase 4: Certificate Revocation

Reasons for Revocation:

1. Private key compromise
2. Certificate information becomes inaccurate
3. Certificate no longer needed
4. CA compromise
5. Affiliation change (employee leaves company)

Revocation Process:

1. Certificate holder or RA requests revocation
2. CA adds certificate to CRL
3. OCSP responder updated
4. Changes propagate to all repositories

5 Implementing PKI Operations in Python

5.1 Example 1: Complete Self-Signed Certificate Creation

```
1 from cryptography import x509
2 from cryptography.x509.oid import NameOID, ExtensionOID
3 from cryptography.hazmat.primitives import hashes
4 from cryptography.hazmat.primitives.asymmetric import rsa
5 from cryptography.hazmat.primitives import serialization
6 from cryptography.hazmat.backends import default_backend
7 import datetime
8
9 # Step 1: Generate RSA key pair
10 print("Generating RSA key pair...")
11 private_key = rsa.generate_private_key(
12     public_exponent=65537,
13     key_size=2048,
14     backend=default_backend()
15 )
16
17 # Step 2: Create certificate subject and issuer (same for self-signed)
18 subject = issuer = x509.Name([
19     x509.NameAttribute(NameOID.COUNTRY_NAME, u"PK"),
20     x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"Punjab"),
21     x509.NameAttribute(NameOID.LOCALITY_NAME, u"Rawalpindi"),
22     x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"CS Department"),
23     x509.NameAttribute(NameOID.COMMON_NAME, u"student.example.edu.pk"),
24 ])
25
```

```

26 # Step 3: Build the certificate
27 cert = x509.CertificateBuilder().subject_name(
28     subject
29 ).issuer_name(
30     issuer
31 ).public_key(
32     private_key.public_key() # Add public key to certificate
33 ).serial_number(
34     x509.random_serial_number() # Generate unique serial number
35 ).not_valid_before(
36     datetime.datetime.utcnow() # Certificate valid from now
37 ).not_valid_after(
38     datetime.datetime.utcnow() + datetime.timedelta(days=365) # Valid for 1
39     year
40 ).add_extension(
41     # Add Subject Alternative Names (SANs)
42     x509.SubjectAlternativeName([
43         x509.DNSName(u"student.example.edu.pk"),
44         x509.DNSName(u"www.student.example.edu.pk"),
45     ]),
46     critical=False,
47 ).add_extension(
48     # Add Key Usage extension
49     x509.KeyUsage(
50         digital_signature=True,
51         key_encipherment=True,
52         content_commitment=False,
53         data_encipherment=False,
54         key_agreement=False,
55         key_cert_sign=False,
56         crl_sign=False,
57         encipher_only=False,
58         decipher_only=False,
59     ),
60     critical=True,
61 ).sign(private_key, hashes.SHA256(), default_backend()) # Sign with SHA-256
62
63 # Step 4: Save private key to file
64 with open("private_key.pem", "wb") as f:
65     f.write(private_key.private_bytes(
66         encoding=serialization.Encoding.PEM,
67         format=serialization.PrivateFormat.TraditionalOpenSSL,
68         encryption_algorithm=serialization.BestAvailableEncryption(b"
69         mypassword")
70     ))
71
72 # Step 5: Save certificate to file
73 with open("certificate.pem", "wb") as f:
74     f.write(cert.public_bytes(serialization.Encoding.PEM))
75
76 print("Certificate created successfully!")
77 print(f"Serial Number: {cert.serial_number}")

```

```

76 print(f"Valid From: {cert.not_valid_before}")
77 print(f"Valid Until: {cert.not_valid_after}")

```

Listing 1: Generate Self-Signed X.509 Certificate

5.2 Example 2: Certificate Validation

```

1 from cryptography import x509
2 from cryptography.hazmat.backends import default_backend
3 import datetime
4
5 # Load certificate from file
6 with open("certificate.pem", "rb") as f:
7     cert_data = f.read()
8     cert = x509.load_pem_x509_certificate(cert_data, default_backend())
9
10 print("=== Certificate Validation ===\n")
11
12 # 1. Extract and display subject information
13 print("Subject Information:")
14 for attribute in cert.subject:
15     print(f"    {attribute.oid._name}: {attribute.value}")
16
17 # 2. Extract and display issuer information
18 print("\nIssuer Information:")
19 for attribute in cert.issuer:
20     print(f"    {attribute.oid._name}: {attribute.value}")
21
22 # 3. Check validity period
23 current_time = datetime.datetime.utcnow()
24 print(f"\nValidity Period:")
25 print(f"    Not Before: {cert.not_valid_before}")
26 print(f"    Not After: {cert.not_valid_after}")
27 print(f"    Current Time: {current_time}")
28
29 # Validate time
30 if cert.not_valid_before <= current_time <= cert.not_valid_after:
31     print("Certificate is currently valid")
32 else:
33     print("Certificate is EXPIRED or not yet valid")
34
35 # 4. Display serial number
36 print(f"\nSerial Number: {cert.serial_number}")
37
38 # 5. Display signature algorithm
39 print(f"\nSignature Algorithm: {cert.signature_algorithm_oid._name}")
40
41 # 6. Extract extensions
42 print("\nCertificate Extensions:")
43 for extension in cert.extensions:
44     print(f"    - {extension.oid._name}")

```

```

45     if extension.oid._name == "subjectAltName":
46         print(f"        SANS: {extension.value}")
47     elif extension.oid._name == "keyUsage":
48         print(f"        Key Usage: {extension.value}")
49
50 # 7. Verify if self-signed
51 if cert.issuer == cert.subject:
52     print("\nThis is a self-signed certificate")
53 else:
54     print("\nThis certificate is signed by a CA")
55
56 print("\n=== Validation Complete ===")

```

Listing 2: Validate Certificate Properties

5.3 Example 3: Create Certificate Revocation List (CRL)

```

1 from cryptography import x509
2 from cryptography.x509.oid import NameOID
3 from cryptography.hazmat.primitives import hashes, serialization
4 from cryptography.hazmat.primitives.asymmetric import rsa
5 from cryptography.hazmat.backends import default_backend
6 import datetime
7
8 # Load CA's private key (assuming we have one)
9 # For demo, we'll generate a new CA key
10 ca_private_key = rsa.generate_private_key(
11     public_exponent=65537,
12     key_size=2048,
13     backend=default_backend()
14 )
15
16 # Create CA name
17 ca_name = x509.Name([
18     x509.NameAttribute(NameOID.COUNTRY_NAME, u"PK"),
19     x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"Example CA"),
20     x509.NameAttribute(NameOID.COMMON_NAME, u"Example CA Root"),
21 ])
22
23 # Build CRL with revoked certificates
24 builder = x509.CertificateRevocationListBuilder()
25 builder = builder.issuer_name(ca_name)
26 builder = builder.last_update(datetime.datetime.utcnow())
27 builder = builder.next_update(datetime.datetime.utcnow() + datetime.timedelta(
28     days=7))
29
30 # Add revoked certificates
31 # Certificate 1: Revoked due to key compromise
32 revoked_cert_1 = x509.RevokedCertificateBuilder().serial_number(
33     12345 # Serial number of revoked certificate
34 ).revocation_date(

```

```

34     datetime.datetime.utcnow() - datetime.timedelta(days=2)
35 ).add_extension(
36     x509.CRLReason(x509.ReasonFlags.key_compromise), # Revocation reason
37     critical=False
38 ).build(default_backend())
39
40 # Certificate 2: Revoked due to cessation of operation
41 revoked_cert_2 = x509.RevokedCertificateBuilder().serial_number(
42     67890 # Another revoked certificate
43 ).revocation_date(
44     datetime.datetime.utcnow() - datetime.timedelta(days=5)
45 ).add_extension(
46     x509.CRLReason(x509.ReasonFlags.cessation_of_operation),
47     critical=False
48 ).build(default_backend())
49
50 # Add revoked certificates to CRL
51 builder = builder.add_revoked_certificate(revoked_cert_1)
52 builder = builder.add_revoked_certificate(revoked_cert_2)
53
54 # Sign the CRL
55 crl = builder.sign(
56     private_key=ca_private_key,
57     algorithm=hashes.SHA256(),
58     backend=default_backend()
59 )
60
61 # Save CRL to file
62 with open("crl.pem", "wb") as f:
63     f.write(crl.public_bytes(serialization.Encoding.PEM))
64
65 print("CRL created successfully!")
66 print(f"Issuer: {crl.issuer}")
67 print(f>Last Update: {crl.last_update}")
68 print(f"Next Update: {crl.next_update}")
69 print(f"Number of revoked certificates: {len(list(crl))}")
70
71 # Function to check if a certificate is revoked
72 def is_certificate_revoked(serial_number, crl_file="crl.pem"):
73     """Check if a certificate serial number is in the CRL"""
74     with open(crl_file, "rb") as f:
75         crl_data = f.read()
76         crl = x509.load_pem_x509_crl(crl_data, default_backend())
77
78     for revoked_cert in crl:
79         if revoked_cert.serial_number == serial_number:
80             return True, revoked_cert.revocation_date
81     return False, None
82
83 # Test the function
84 print("\n=== Testing Certificate Revocation Status ===")
85 test_serials = [12345, 67890, 99999]

```

```

86 for serial in test_serials:
87     is_revoked, revocation_date = is_certificate_revoked(serial)
88     if is_revoked:
89         print(f"Certificate {serial}: REVOKED on {revocation_date}")
90     else:
91         print(f"Certificate {serial}: VALID (not revoked)")

```

Listing 3: Generate and Manage CRL

6 PKI in Real-World Scenarios

6.1 Scenario 1: HTTPS/TLS in Web Browsers

How it Works:

1. User enters `https://example.com` in browser
2. Browser initiates TLS handshake with server
3. Server sends its certificate to browser
4. Browser validates certificate:
 - Checks signature using CA's public key
 - Verifies certificate chain to trusted root
 - Checks expiration date
 - Queries OCSP or checks CRL
 - Verifies domain name matches certificate
5. If valid, browser displays padlock icon
6. Encrypted session established using symmetric encryption

Security Indicators:

- **Green Padlock:** Valid certificate, encrypted connection
- **Warning/Red:** Invalid certificate (expired, untrusted CA, name mismatch)
- **Extended Validation (EV):** Company name shown in address bar (stricter validation)

6.2 Scenario 2: Email Security with S/MIME

Components:

- Each user has a certificate with their email address
- Private key used to sign emails
- Public key used to encrypt emails to the recipient

Operations:

1. **Signing Email:**

- Alice signs email with her private key
- Bob verifies signature using Alice's public key from her certificate
- Provides authentication and integrity

2. **Encrypting Email:**

- Alice encrypts email using Bob's public key from his certificate
- Only Bob can decrypt with his private key
- Provides confidentiality

6.3 Scenario 3: Code Signing

Purpose: Verify software publisher and ensure code hasn't been modified.

Process:

1. Developer obtains code signing certificate from CA
2. Developer signs software package/executable with private key
3. User downloads software
4. Operating system verifies signature using developer's certificate
5. If valid, OS trusts the software

Benefits:

- Users know who created the software
- Malware cannot impersonate legitimate publishers
- Operating systems can enforce security policies (e.g., block unsigned code)

7 PKI Security Considerations

7.1 Threats to PKI

1. **CA Compromise:**

- If CA's private key is stolen, attacker can issue fraudulent certificates
- *Mitigation:* HSMs, key ceremonies, audit trails, certificate pinning

2. **Man-in-the-Middle Attacks:**

- Attacker presents fraudulent certificate
- *Mitigation:* Certificate pinning, CT logs

3. **Key Compromise:**

- End entity's private key stolen

- *Mitigation:* Immediate revocation, short certificate lifetimes

4. Social Engineering:

- Trick CA/RA into issuing certificate to wrong entity
- *Mitigation:* Strict validation procedures, multi-factor authentication

7.2 Best Practices

- **Use Strong Algorithms:** RSA 2048-bit minimum, prefer ECDSA P-256 or higher
- **Short Certificate Lifetimes:** 90 days reduces window of exposure
- **Automated Renewal:** Use ACME protocol (Let's Encrypt)
- **Certificate Transparency:** Publicly log all certificates
- **HSTS (HTTP Strict Transport Security):** Force HTTPS connections
- **Certificate Pinning:** Hard-code expected certificates in applications
- **Regular Audits:** Monitor certificate usage and expiration
- **Secure Key Storage:** Use HSMs for CA keys, encrypted storage for end entities

8 Practice Questions and Activities

8.1 Programming Exercises

Exercise 1: Certificate Information Extractor

Write a Python program that:

- Loads a certificate from a file
- Extracts and displays all fields in a formatted manner
- Validates the certificate's validity period
- Checks if it's self-signed or CA-signed
- Displays all extensions with their values

Exercise 2: Certificate Chain Validator

Implement a certificate chain validation function that:

- Takes a list of certificates (end-entity $\rightarrow$ intermediate $\rightarrow$ root)
- Verifies each certificate's signature using the next certificate's public key
- Checks that each certificate's issuer matches the next certificate's subject
- Validates the entire chain to the root
- Returns True/False with detailed error messages

Exercise 3: Simple PKI Simulator

Create a complete PKI system simulation:

- Generate a root CA certificate (self-signed)
- Generate an intermediate CA certificate (signed by root)
- Generate 3 end-entity certificates (signed by intermediate)
- Create a CRL with one revoked certificate
- Implement certificate validation that checks the chain and CRL
- Test the entire system with valid and revoked certificates

Starter Code for Exercise 3:

```
1 from cryptography import x509
2 from cryptography.hazmat.primitives.asymmetric import rsa
3 from cryptography.hazmat.primitives import hashes, serialization
4 from cryptography.hazmat.backends import default_backend
5 from cryptography.x509.oid import NameOID
6 import datetime
7
8 class SimplePKI:
9     def __init__(self):
10         self.root_ca_key = None
11         self.root_ca_cert = None
12         self.intermediate_ca_key = None
13         self.intermediate_ca_cert = None
14         self.crl = None
15
16     def create_root_ca(self):
17         """Generate root CA key pair and self-signed certificate"""
18         # TODO: Implement root CA creation
19         pass
20
21     def create_intermediate_ca(self):
22         """Generate intermediate CA and sign with root CA"""
23         # TODO: Implement intermediate CA creation
24         pass
25
26     def issue_certificate(self, common_name, organization):
27         """Issue end-entity certificate signed by intermediate CA"""
28         # TODO: Implement certificate issuance
29         pass
30
31     def create_crl(self, revoked_serials):
32         """Create CRL with list of revoked certificate serial numbers"""
33         # TODO: Implement CRL creation
34         pass
35
36     def validate_certificate(self, cert):
37         """Validate certificate chain and check CRL"""
```

```

38     # TODO: Implement validation
39     # 1. Verify signature chain
40     # 2. Check validity period
41     # 3. Check CRL
42     pass
43
44 # Test the PKI system
45 if __name__ == "__main__":
46     pki = SimplePKI()
47
48     # Create CA hierarchy
49     pki.create_root_ca()
50     pki.create_intermediate_ca()
51
52     # Issue certificates
53     cert1 = pki.issue_certificate("student1.edu.pk", "CS Department")
54     cert2 = pki.issue_certificate("student2.edu.pk", "CS Department")
55     cert3 = pki.issue_certificate("student3.edu.pk", "CS Department")
56
57     # Revoke one certificate
58     pki.create_crl([cert2.serial_number])
59
60     # Validate certificates
61     print("Certificate 1:", pki.validate_certificate(cert1))
62     print("Certificate 2:", pki.validate_certificate(cert2)) # Should fail
63     print("Certificate 3:", pki.validate_certificate(cert3))

```

Listing 4: PKI Simulator Template

8.2 Lab Activity: HTTPS Certificate Analysis

Objective: Analyze real-world PKI implementation in web browsers.

Instructions:

1. Open your web browser and navigate to <https://www.google.com>
2. Click on the padlock icon and view certificate details
3. Document the following:
 - Subject (Common Name, Organization)
 - Issuer (CA name)
 - Validity period (Not Before, Not After)
 - Serial number
 - Signature algorithm
 - Public key algorithm and size
 - Extensions (Key Usage, Subject Alternative Names, etc.)
4. Examine the certificate chain:

- How many certificates are in the chain?
- What is the root CA?
- What intermediate CAs are present?

5. Use Python to fetch and analyze the certificate:

```

1 import ssl
2 import socket
3 from cryptography import x509
4 from cryptography.hazmat.backends import default_backend
5
6 def get_server_certificate(hostname, port=443):
7     """Fetch certificate from HTTPS server"""
8     context = ssl.create_default_context()
9
10    with socket.create_connection((hostname, port)) as sock:
11        with context.wrap_socket(sock, server_hostname=hostname) as ssock:
12            # Get certificate in DER format
13            der_cert = ssock.getpeercert(binary_form=True)
14
15            # Load certificate
16            cert = x509.load_der_x509_certificate(der_cert, default_backend())
17
18            return cert
19
20 # Fetch and analyze certificate
21 hostname = "www.google.com"
22 cert = get_server_certificate(hostname)
23
24 print(f"=== Certificate Analysis for {hostname} ===\n")
25 print(f"Subject: {cert.subject}")
26 print(f"Issuer: {cert.issuer}")
27 print(f"Serial Number: {cert.serial_number}")
28 print(f"Valid From: {cert.not_valid_before}")
29 print(f"Valid Until: {cert.not_valid_after}")
30 print(f"Signature Algorithm: {cert.signature_algorithm_oid._name}")
31
32 # Check if certificate is currently valid
33 import datetime
34 now = datetime.datetime.utcnow()
35 if cert.not_valid_before <= now <= cert.not_valid_after:
36     print("\nCertificate is VALID")
37 else:
38     print("\nCertificate is EXPIRED or not yet valid")
39
40 # Display Subject Alternative Names
41 try:
42     san_ext = cert.extensions.get_extension_for_oid(
43         x509.oid.ExtensionOID.SUBJECT_ALTERNATIVE_NAME
44     )
45     print(f"\nSubject Alternative Names:")

```

```

46     for name in san_ext.value:
47         print(f"    - {name.value}")
48 except x509.ExtensionNotFound:
49     print("\nNo Subject Alternative Names found")

```

Listing 5: Fetch Certificate from HTTPS Server

Analysis Questions:

1. Why does Google's certificate include multiple Subject Alternative Names?
2. What is the validity period of the certificate? Why is it relatively short?
3. Research the root CA. Is it trusted by your operating system/browser?
4. What would happen if the certificate expired? Test with an expired certificate website.

9 Introduction to Public-Key Certificates

9.1 What is a Public-Key Certificate?

A **public-key certificate** acts as a digital passport that:

1. Binds a public key to a specific identity (person, organization, or device)
2. Is issued and digitally signed by a trusted third party called a **Certificate Authority (CA)**
3. Allows users to verify the authenticity of a public key without direct interaction

9.2 Certificate Structure Notation

The formal notation for a certificate is:

$$\text{Cert-CKCA}(+KA) = CA[V, SN, AI, CA, TCA, A, +KA]$$

Components Explained:

V: Version number of the certificate format

SN: Serial number (unique identifier for the certificate)

AI: Algorithm identifier (e.g., RSA with SHA-256)

CA: Name of the Certificate Authority

TCA: Period of validity (start and expiration dates)

A: Name of the entity (user/organization)

+KA: Public key being certified

-CKCA: CA's private key (used to sign the certificate)

9.3 Key Properties of Certificates

1. **Trust Model:** All users must trust the CA's public key
2. **No Online TTP Required:** Certificates can be verified offline
3. **Security Dependency:** The CA's private key security is critical

10 Certificate Chains and Hierarchy

10.1 The Trust Problem

Scenario: Alice and Bob live in different regions and have certificates from different CAs:

- Alice has a certificate from CAA
- Bob has a certificate from CAB

Problem: If Alice doesn't trust CAB, Bob's certificate is useless to her, and vice versa.

10.2 Solution: Certificate Chains

A **certificate chain** is a sequence of certificates where each certificate certifies the public key of the next CA in the chain.

Example Chain:

CAA<<CAB>>, CAB<<Bob>>

How it Works:

1. CAA issues a certificate for CAB's public key
2. CAB issues a certificate for Bob's public key
3. Alice trusts CAA, so she can verify the entire chain

Note: Certificate chains can have multiple levels (not just two).

11 Implementing X.509 Certificates in Python

11.1 Step 1: Generating an RSA Private Key

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives import serialization
3 from cryptography.hazmat.primitives.asymmetric import rsa
4
5 # Define encryption password for the private key
6 encrypted_pass = b"Ilik32Cod3"
7
8 # Generate RSA key pair (2048-bit key, public exponent 65537)
9 key = rsa.generate_private_key(
10     public_exponent=65537,
11     key_size=2048,
```

```

12     backend=default_backend()
13 )
14
15 # Save the private key to a PEM file with encryption
16 with open("rsakey.pem", "wb") as f:
17     f.write(key.private_bytes(
18         encoding=serialization.Encoding.PEM,
19         format=serialization.PrivateFormat.TraditionalOpenSSL,
20         encryption_algorithm=serialization.BestAvailableEncryption(
21             encrypted_pass)
22     ))
23 print("Private key generated and saved to rsakey.pem")

```

Listing 6: Generate and Save Encrypted Private Key

Key Points:

- RSA with 2048-bit key size provides strong security
- Private key is encrypted using AES (via BestAvailableEncryption)
- Public exponent 65537 is the standard choice

11.2 Step 2: Creating a Certificate Signing Request (CSR)

```

1 from cryptography import x509
2 from cryptography.x509.oid import NameOID
3 from cryptography.hazmat.primitives import hashes
4
5 # Build the CSR with subject details
6 csr = x509.CertificateSigningRequestBuilder().subject_name(
7     x509.Name([
8         x509.NameAttribute(NameOID.COUNTRY_NAME, u"US"),
9         x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"NC"),
10        x509.NameAttribute(NameOID.LOCALITY_NAME, u"Raleigh"),
11        x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"PythonCryptography"),
12        x509.NameAttribute(NameOID.COMMON_NAME, u"shannonbray.us"),
13    ])
14 ).add_extension(
15     # Add alternative DNS names
16     x509.SubjectAlternativeName([
17         x509.DNSName(u"shannonbray.us"),
18     ]),
19     critical=False,
20     # Sign the CSR with the private key using SHA-256
21 ).sign(key, hashes.SHA256(), default_backend())
22
23 # Save CSR to file
24 with open("csr.pem", "wb") as f:
25     f.write(csr.public_bytes(serialization.Encoding.PEM))
26
27 print("CSR generated and saved to csr.pem")

```

Listing 7: Generate CSR with Subject Information

CSR Components:

- **Subject Name:** Identity information (Country, State, Organization, etc.)
- **Subject Alternative Name:** Additional DNS names for the certificate
- **Signature:** CSR is signed with the private key to prove ownership

11.3 Step 3: Loading an Existing Private Key

```
1 from cryptography.hazmat.primitives.serialization import load_pem_private_key
2
3 # Define the password
4 encrypted_pass = b"Ilik32Cod3"
5
6 # Load the private key from the PEM file
7 with open('rsakey.pem', 'rb') as key_file:
8     private_key = load_pem_private_key(
9         key_file.read(),
10        password=encrypted_pass,
11        backend=default_backend()
12    )
13
14 print("Private key loaded successfully")
```

Listing 8: Load Encrypted Private Key from File

12 Certificate Revocation

12.1 Why Revocation is Necessary

Problem: If Alice's private key is compromised:

- An attacker (Trent) can impersonate Alice
- The compromise persists until the certificate expires
- Alice needs a way to invalidate her certificate immediately

12.2 Certificate Revocation Lists (CRLs)

A **CRL** is a list of certificates that have been revoked before their expiration date.

CRL Process:

1. When a certificate is compromised, the owner requests revocation
2. The CA adds the certificate's serial number to the CRL
3. The CRL is published (typically in an X.500 directory)

4. Before accepting a certificate, users must check:

- That the certificate is valid
- That it is NOT on the CRL

Limitations of CRLs:

- Slow propagation time
- Network overhead
- Costly to maintain
- Potential for stale CRLs

Modern Alternatives:

- **OCSP (Online Certificate Status Protocol):** Real-time certificate status checking
- **OCSP Stapling:** Server includes OCSP response with certificate

13 Practice Lab Activities

13.1 Lab Activity 1: Certificate Validation

Objective: Validate a self-signed certificate programmatically.

Task: Write a Python script that:

1. Loads a self-signed certificate from a file
2. Extracts certificate details (subject, issuer, validity period)
3. Verifies the certificate signature
4. Displays the validation results

Hints:

- Use `x509.load_pem_x509_certificate()`
- Access certificate attributes like `subject`, `issuer`, `not_valid_before`
- Check if the current date is within the validity period

13.2 Lab Activity 2: Certificate Revocation List

Objective: Create and manage a simple Certificate Revocation List.

Task: Implement a CRL system that:

1. Creates a CRL builder
2. Adds revoked certificates with serial numbers and revocation dates
3. Signs the CRL with a CA's private key

4. Saves the CRL to a file
5. Loads and parses the CRL to check if a certificate is revoked

Hints:

- Use `x509.CertificateRevocationListBuilder()`
- Add revoked certificates using `add_revoked_certificate()`
- Sign with CA private key using `sign()`
- Check revocation status by comparing serial numbers